Ch. 3 Cheat Sheet

Data Structures, Big-O, Sorting

ADT vs. data structure

ADT = interface (*what*). **Data structure** = implementation (*how*). One ADT, many implementations.

ADT	**typical DS**
List	array, linked list
Stack	array / linked list
Queue	linked list / ring buf
Priority queue	binary heap
Set	hash table / BST
Map / Dictionary	hash table / BST
Graph	adj. list / matrix

The seven structures

- **Array / vector** – contiguous; $O(1)$ indexing.
- **Linked list** – nodes + pointers; $O(1)$ insert if you hold the prev.
- **Tree** – hierarchical; recursion natural.
- **BST** – ordered tree; $O(\log n)$ ops if balanced.
- **Heap** – complete binary tree; $O(1)$ peek, $O(\log n)$ push/pop.
- **Hash table** – bucketed; $O(1)$ expected ops.
- **Graph** – nodes + edges; adjacency list or matrix.

Searching

Linear search	any data	$O(n)$
Binary search	sorted	$O(\log n)$

Binary search midpoint: `lo + (hi - lo) / 2` (avoid overflow).

Big-O in one sentence

$f(n) = O(g(n))$ iff $\exists c > 0, n_0$ such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. Drop constants, drop lower-order terms, keep the dominant term.

Growth hierarchy

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

Rule of thumb for loops:

- Single loop over n: $O(n)$.
- Nested over n: $O(n^2)$.
- Halving each iteration: $O(\log n)$.
- Divide-and-conquer ($T(n) = 2T(n/2) + O(n)$): $O(n \log n)$.

Constant-time operations

`v[i]`, `v.back()`, `push_back` (amortized), `pop_back`, `size()`, arithmetic, a bounded-iteration loop. *Not* constant: hashing a long string, a loop whose bound depends on input.

Sorting comparison

Sort	best	avg	worst	stable	in-place
Bubble	n	n^2	n^2	yes	yes
Insertion	n	n^2	n^2	yes	yes
Selection	n^2	n^2	n^2	no	yes
Shell	$n \log n$	var.	$n^{3/2}$	no	yes
Merge	$n \log n$	$n \log n$	$n \log n$	yes	no
Quick	$n \log n$	$n \log n$	n^2	no	yes
Heap	$n \log n$	$n \log n$	$n \log n$	no	yes
Radix (LSD)	nk	nk	nk	yes	no

"Which sort do I pick?"

- **Small array ($n < 20$) or nearly sorted**: insertion sort ($O(n)$ best case, tiny constants).
- **General-purpose, RAM-bound**: quicksort (what `std::sort` uses, with heapsort fallback).
- **Stability required**: merge sort (`std::stable_sort`).
- **External / too big for RAM**: merge sort (sequential I/O; the model for external mergesort).
- **Integer keys, bounded width**: radix sort (beats comparison-based $\Omega(n \log n)$ bound).
- **Tightest worst-case guarantee**: heap sort.

The $\Omega(n \log n)$ lower bound

Any *comparison-based* sort needs $\Omega(n \log n)$ comparisons in the worst case. Decision-tree argument: $n!$ possible outputs, so any binary tree that distinguishes them has depth $\geq \log_2(n!) = \Theta(n \log n)$. Radix and counting sort beat this because they're *non-comparison*.

Stable vs. unstable

A sort is **stable** if equal keys retain their original relative order. Matters when sorting by one field after another (e.g., sort by last name, then stable-sort by city).

In-place

Uses $O(1)$ or $O(\log n)$ auxiliary memory. Merge sort's $O(n)$ buffer makes it *not* in-place; its recursion stack is $O(\log n)$.

Quicksort: watch the pivot

Average $O(n \log n)$; worst $O(n^2)$ on already-sorted input with first-element pivot. Fixes: median-of-three; randomize. `std::sort` uses introsort: quicksort + fallback to heapsort after a depth limit.

Top gotchas

- Quoting "$O(n \log n)$ always" for quicksort (it's average; worst case is $O(n^2)$).
- Saying merge sort is in-place (it's not).
- Using `strcmp`-style comparators for numeric sort.
- Assuming `std::sort` is stable (use `std::stable_sort`).
- Reasoning about "fast" without bounding n.